

Sistemes Distribuïts en Xarxa (SDX)
Facultat d'Informàtica de Barcelona
Control parcial. 21 d'Abril 2021

Contesteu a les preguntes de manera concisa i precisa
Contesteu al mateix full
Test, seminaris: no es poden consultar apunts
Readings: podeu portar els vostres informes
Problemes: podeu portar un full de notes imprès a doble cara
Durada: 1 hora, 40 minuts

Nom i Cognoms:

1. (5 pts) Seleccioneu la resposta (una només) que considereu correcta en cadascun dels apartats. Cada resposta correcta val 1/2 punts. Cada resposta incorrecta resta 1/6.
 - (a) Quina de les següents tècniques NO millora l'escalabilitat geogràfica dels sistemes distribuïts?
 - ☐ Fer servir comunicació asíncrona per amagar la latència de xarxa
 - ☐ Replicar servidors a prop d'on es troben els clients
 - ☐ Fer caching de les dades en els clients
 - ☐ Augmentar la capacitat dels nodes (usar CPUs més ràpides, afegir més memòria, ...)
 - (b) Una aplicació distribuïda utilitza un servei de consulta de la data i hora oficials proporcionat pel Ministeri d'Educació mitjançant la URL <https://sede.educacion.gob.es/normativa/fecha-hora.html> i que està suportat per un grup de servidors idèntics que és capaç de oferir un servei correcte tot i que algun servidor estigui caigut o retorni valors incorrectes. Per motius polítics, el govern decideix traslladar aquest servei al Ministeri de Foment i fer-lo accessible mitjançant la URL <https://sede.mitma.gob.es/MFOM.FechaHora/>. Des de la perspectiva de l'aplicació distribuïda, quin dels següents tipus de transparència NO està proporcionant aquest sistema?
 - ☐ Location transparency
 - ☐ Mobility transparency
 - ☐ Replication transparency
 - ☐ Failure transparency
 - (c) Quina de les següents afirmacions referents a la comunicació síncrona en sistemes distribuïts és certa?
 - ☐ Les operacions *send* i *receive* són bloquejants
 - ☐ L'operació *send* és bloquejant i l'operació *receive* és no-bloquejant
 - ☐ L'operació *send* és no-bloquejant i l'operació *receive* és bloquejant
 - ☐ Les operacions *send* i *receive* són no-bloquejants
 - (d) Un client que està comprant en una botiga online fa una invocació remota per afegir un producte al seu cistell. Quina semàntica mínima és necessària per assegurar que s'afegeix al cistell una unitat del producte si es poden perdre missatges però el servidor no té fallades?
 - ☐ Maybe semantics
 - ☐ At-least-once semantics
 - ☐ At-most-once semantics
 - ☐ Exactly-once semantics

- (e) Quin dels següents protocols de *routing* d'events en sistemes de publicació/subscripció envia els events publicats només als nodes veïns mitjançant els quals es poden fer arribar als subscriptors corresponents?
- ☐ Filter-based routing
 - ☐ Gossip-based routing
 - ☐ Subscription flooding routing
 - ☐ Event flooding routing
- (f) Tenim un conjunt de quatre nodes que volen sincronitzar el seu rellotge mitjançant l'algorisme de Berkeley. El temps actual del node master és 3:52:20.750, i el temps dels altres nodes és 3:52:20.150, 3:52:20.800 i 3:52:21.000, respectivament. Si la màxima desviació dels rellotges permesa en aquest sistema és 0:00:00.500, a quin valor fixarà cada node el seu rellotge? (pots ignorar la latència dels missatges)
- ☐ 3:52:20.650
 - ☐ 3:52:20.675
 - ☐ 3:52:20.850
 - ☐ 3:52:20.900
- (g) Tenim un grup de sis processos amb identificadors de P_1 fins a P_6 . El líder actual P_6 té una fallada de crash i el procés P_2 inicia una elecció mitjançant l'algorisme en anell millorat (i.e. *enhanced ring algorithm*). Quin és el comportament de P_3 si rep un missatge **election** que inclou els identificadors $[P_2, P_3, P_4, P_5, P_1]$?
- ☐ Afegeix el seu identificador a la llista i reenvia el missatge **election** al seu successor
 - ☐ Envia un missatge **coordinator** al seu successor proposant P_5 com a nou coordinador
 - ☐ Envia un missatge **coordinator** al seu successor proposant-se a si mateix com a nou coordinador
 - ☐ És absolutament impossible que P_3 rebi aquest missatge, perquè P_2 l'hauria rebut abans i hauria enviat el missatge **coordinator** corresponent
- (h) Un grup de N processos que escull un líder mitjançant l'algorisme *Chang and Roberts* ha necessitat $2N + 1$ missatges per concloure l'elecció. Quin procés va iniciar l'elecció?
- ☐ El procés amb l'identificador més gran
 - ☐ El successor del procés amb l'identificador més gran
 - ☐ El predecessor del procés amb l'identificador més gran
 - ☐ Podria haver estat qualsevol procés, perquè aquest algorisme sempre requereix $2N + 1$ missatges independentment de qui inicia l'elecció
- (i) Donat un conjunt de processos (P_1, P_2, P_3) que es comunica mitjançant multicast causalment ordenat (i.e. *causally-ordered multicast*), si P_1 rep un missatge m procedent de P_3 i el rellotge vectorial de P_1 és $VC_1 = (4, 12, 26)$, quin rellotge vectorial hauria d'incloure m per tal que P_1 pogués lliurar-lo a l'aplicació immediatament?
- ☐ (4, 11, 27)
 - ☐ (4, 10, 28)
 - ☐ (5, 13, 27)
 - ☐ (3, 11, 28)
- (j) El model de consistència *Writes Follow Reads* es pot implementar assegurant que totes les operacions incloses al *read set* s'han dut a terme a la rèplica local abans de procedir amb una operació d'escriptura. Què conté el *read set*?
- ☐ Les operacions de lectura realitzades pel client que està invocant l'operació d'escriptura
 - ☐ Les operacions de lectura realitzades per tots els clients
 - ☐ Les operacions d'escriptura realitzades pel client que està invocant l'operació d'escriptura
 - ☐ Les operacions d'escriptura els efectes de les quals han estat vistos per les operacions de lectura realitzades pel client que està invocant l'operació d'escriptura

Nom i Cognoms:

2. (SEMINARIS) Contesta les següents preguntes referents als seminaris **Muty** i **Chatty**.

- a) (6 pts) **Muty**. Completa el següent extracte de codi corresponent a la implementació del mecanisme d'exclusió mútua de Ricart & Agrawala utilitzant rellotges lògics de Lamport (i.e. *lock3*).

```
% MyClock: Logical clock of this process
% MyReqTime: Timestamp of the request of this process
wait(MyId, Nodes, Master, Refs, Waiting, MyReqTime, MyClock) ->
receive
{request, From, Ref, ReqId, ReqTime} ->
    MyNewClock = ...
    if
        ( ... < ... ) or ((ReqTime == ... ) and (ReqId < ... )) ->
            From ! {ok, Ref},
            wait(MyId, Nodes, Master, Refs, ... , ... , MyNewClock);
        true ->
            wait(MyId, Nodes, Master, Refs, ... , MyReqTime, ... )
    end;
    {ok, Ref} ->
        NewRefs = lists:delete(Ref, ... ),
        wait(MyId, Nodes, Master, NewRefs, Waiting, MyReqTime, ... )
end.
```

- b) (2 pts) **Muty**. El següent extracte de codi inicialitza dos workers com a part d'un sistema d'exclusió mútua. Fes els canvis necessaris per executar cada worker en dues instàncies Erlang separades, 'n1@127.0.0.1' i 'n2@127.0.0.1', respectivament.

```
start(Lock, Sleep, Work) ->
    Main = self(),
    register(w1, worker:start("John", Main, Lock, 1, Sleep, Work)),
    register(w2, worker:start("Ringo", Main, Lock, 2, Sleep, Work)),
    collect(2, []).

stop() ->
    w1 ! stop,
    w2 ! stop.
```

- c) (2 pts) **Chatty**. Què passa quan un servidor té una fallada de crash? Justifica la resposta en funció de si els clients estan connectats a un únic servidor o a múltiples servidors.

3. (SEMINARIS) Contesta les següents preguntes referents al seminari **Ordy**.

- a) (6 pts) **Ordy**. Completa el següent extracte de codi corresponent a la implementació de multicast causalment ordenat.

```
% Check if a message from process Id with clock MsgVC can be delivered to the worker
% Invoked initially as checkMsg(Id, MsgVC, VC, size(VC))
checkMsg(_, _, _, 0) -> ready;
checkMsg(Id, MsgVC, VC, ... ) ->
    if (element(Id, MsgVC) == ... ) ->
        checkMsg(Id, MsgVC, ... , ... );
        true -> wait
    end;
checkMsg(Id, MsgVC, VC, ... ) ->
    if ( ... =< ... ) ->
        checkMsg(Id, MsgVC, ... , ... );
        true -> wait
    end.
```

- b) (2 pts) **Ordy**. Identifica (i justifica) els dos errors presents en el següent extracte de codi corresponent a la implementació de multicast totalment ordenat.

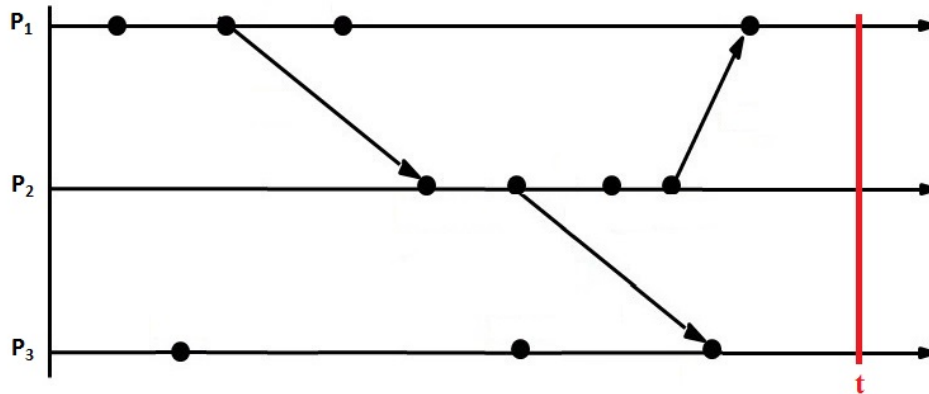
```
server(Master, MaxPrp, MaxAgr, Nodes, Cast, Queue, Jitter) ->
    receive
        {proposal, Ref, Num} ->
            case proposal(Ref, Num, Cast) of
                {agreed, MaxNum, NewCast} ->
                    agree(Ref, Num, Nodes),
                    server(Master, MaxPrp, MaxNum, Nodes, NewCast, Queue, Jitter);
            NewCast ->
                server(Master, MaxPrp, MaxAgr, Nodes, NewCast, Queue, Jitter)
        end;
```

- c) (2 pts) Assumint que tenim N processos worker, indica quants (i quins) missatges es generen en la implementació de multicast totalment ordenat des de que un worker envia un missatge multicast fins que tots els workers l'han lliurat. Compara amb la quantitat de missatges generats en la implementació bàsica de multicast.

Nom i Cognoms:

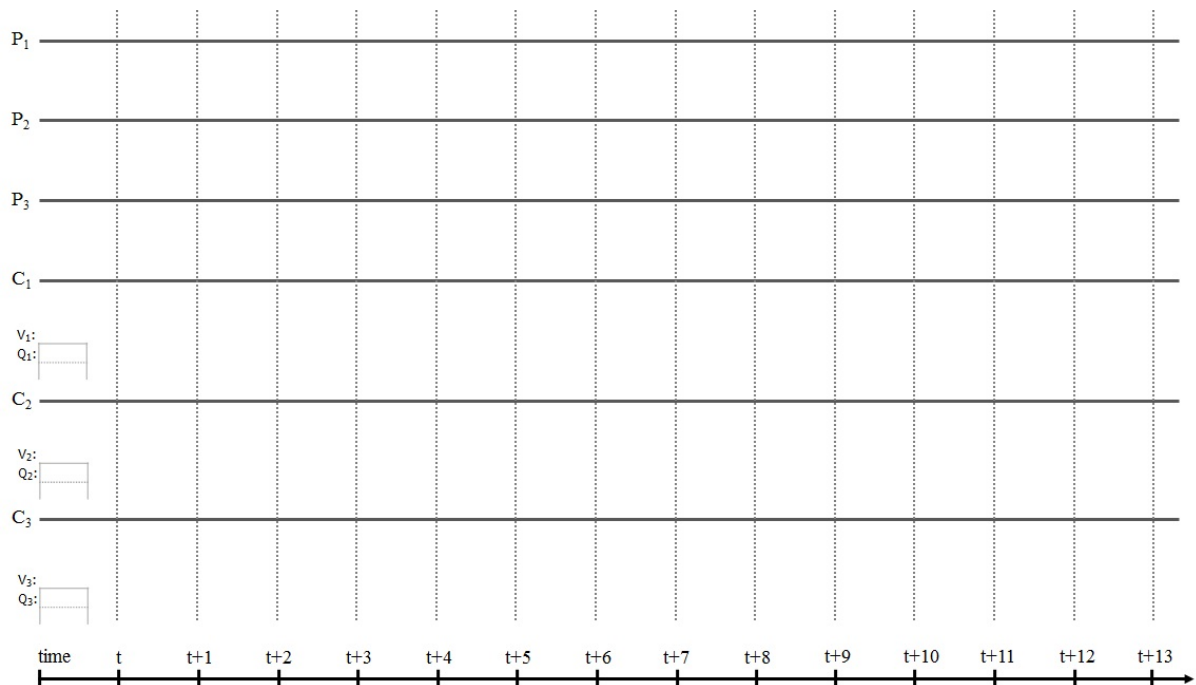
4. (2 pts) Donada la següent seqüència d'events executada pels processos P_1 , P_2 i P_3 .

a) Etiqueta cada event amb el valor del seu rellotge lògic escalar (i.e. rellotge de Lamport).

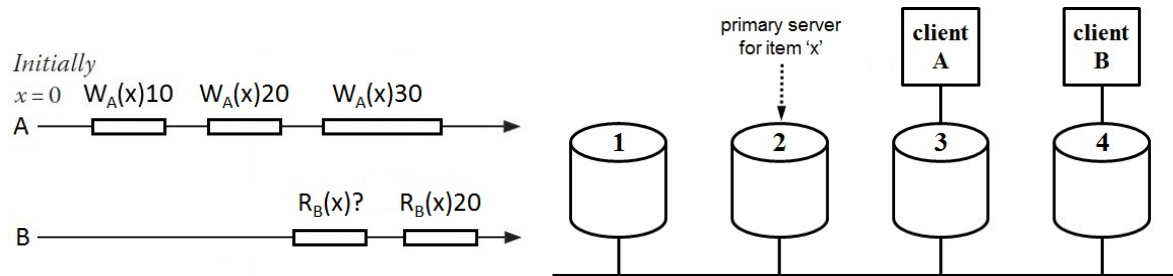


b) A l'instant t , els tres processos anteriors intenten accedir a una regió crítica i per fer-ho es coordinen mitjançant l'algorisme de *Lin*, que està configurat amb tres processos coordinadors (C_1 a C_3) i amb missatges *Yield*. Determina la interacció que hi haurà entre els diferents processos fins que el procés coordinador C_2 rebí un missatge per alliberar la regió crítica (omple el cronograma indicant quins missatges s'envien i es reben en cada instant, i l'estat de cada coordinador: a quin procés ha votat (V_i) i quins processos té a la cua (Q_i)), tenint en compte les latències entre els processos i els coordinadors que es mostren a la taula següent. Nota: quan un procés accedeix a la regió crítica, l'alliberarà passada una unitat de temps.

	C_1	C_2	C_3
P_1	1	3	2
P_2	2	1	3
P_3	3	2	1



5. (1,5 pts) Donats dos clients A i B que executen les operacions tal com es mostren a la figura següent en un *data store* de 4 rèpliques que utilitza un protocol d'escriptura remota basat en primari (i.e. *primary-backup remote-write protocol*) amb escriptura NO bloquejant, indica 1) la seqüència d'accions que es duran a terme per executar les operacions $W_A(x)10$ i $R_B(x)20$, 2) quin possibles valors pot retornar l'operació $R_B(x)?$, si el *data store* proporciona els següents models de consistència:

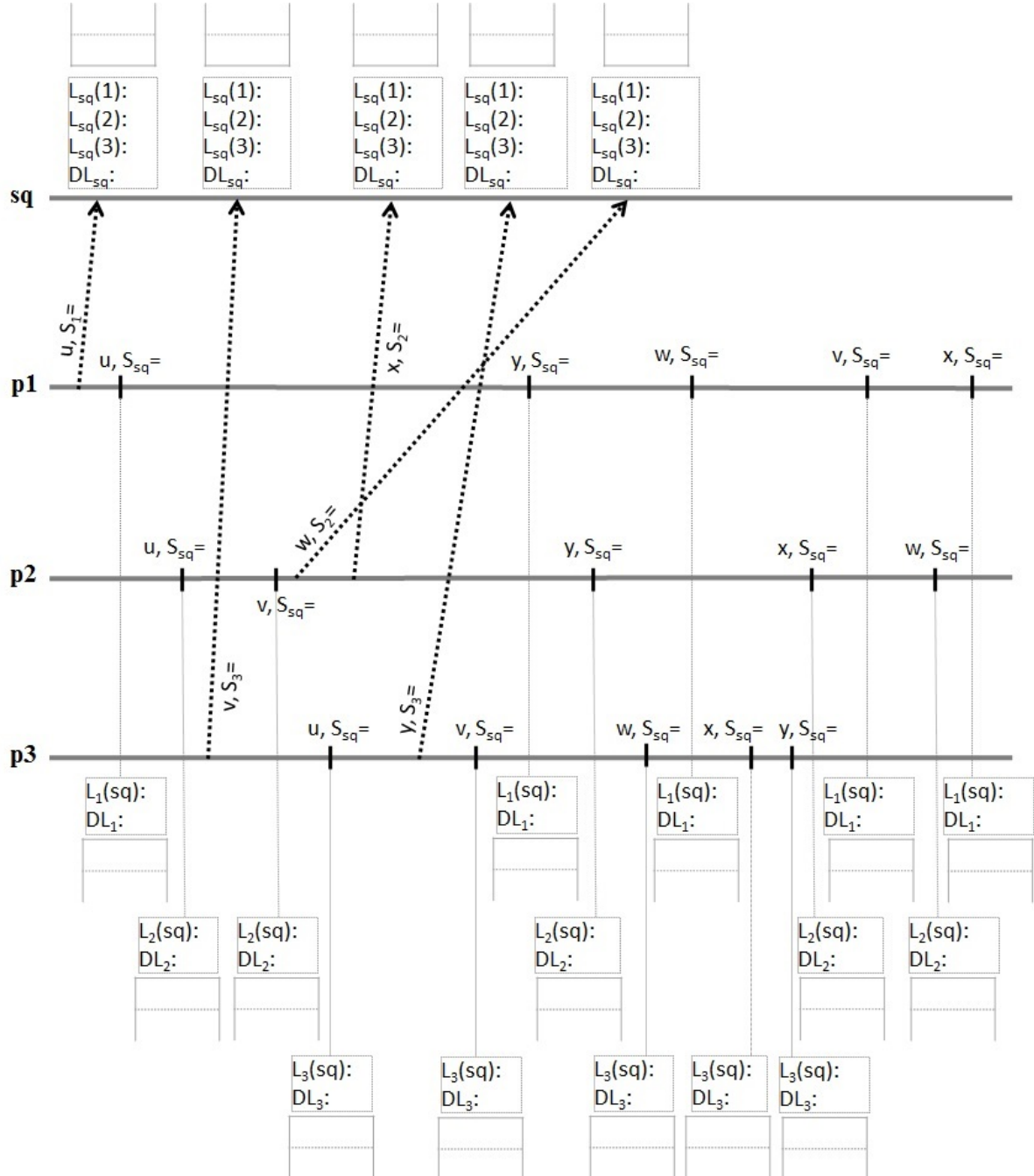


a) Consistència seqüencial.

b) Linearizability.

Nom i Cognoms:

6. (1,5 pts) Tenim el següent grup de processos que es comuniquen mitjançant multicast totalment ordenat (i.e. *totally-ordered multicast*) implementat fent servir un procés seqüenciador (sq). Tenint en compte que els missatges que arriben al procés sq s'han de processar respectant una ordenació FIFO, indica per cada procés p_i o sq : el número de seqüència del darrer missatge que el procés p_i ha lliurat procedent del procés sq ($L_i(sq)$), el número de seqüència del darrer missatge que el procés sq ha processat procedent de qualsevol altre procés i ($L_{sq}(i)$), la llista de missatges lliurats o processats fins el moment (DL_i , DL_{sq}), els missatges guardats a la *hold-back queue* pendents de ser lliurats o processats, i el número de seqüència de cada missatge multicast enviat (S_i , S_{sq}). Nota: la figura indica en quin instant cada procés i rep els missatges procedents del procés sq .



7. (READINGS) Contesta les següents preguntes referents als articles llegits a l'assignatura.
- i) Explica quina problemàtica pot comportar l'enviament de missatges grans entre dos nodes en Erlang segons es descriu a l'article *Hebert13*.
 - ii) Defineix el concepte de *syntonization* i explica quins factors determinen la seva precisió segons l'article *Neville-Neil16*.
 - iii) Indica quan el model *eventual consistency* es comporta de la mateixa manera que el model *consistent prefix* segons es descriu a l'article *Terry13*.